Functional Programming

Lecture 1

Kameliya Golova (she/her)

Constructor University Bremen

Welcome

Plan for today:

- What is functional programming?
- How does this course work?
- Introduction to logic in Lean

What is functional programming?

Places

Let's start simple:

```
int x = 0;
x = 5;
```

Places

Let's start simple:

```
int x = 0;
x = 5;
```

What does this do?

1. Create a "place" x.
2. Store the value 0 in x.
3. Store the value 5 in x.

Places

Let's start simple:

```
int x = 0;
x = 5;
```

What does this do?

1. Create a "place" x.
2. Store the value 0 in x.
3. Store the value 5 in x.

The place x is not the same as the value of x.

Imperative languages create places for each variable.

Places in practice

What can this C++ function return?

```
int subtract_self(int& x, int& y) {
  int z = x;
  y += 1;
  return x - z;
}
```

Places in practice

What can this C++ function return?

```
int subtract_self(int& x, int& y) {
  int z = x;
  y += 1;
  return x - z;
}
```

- If x and y are different places, 0.
- If x and y are the same place, 1.

What if we... don't?

Functional programming languages do not create places for you.

```
def x := 0
```

What if we... don't?

Functional programming languages do not create places for you.

```
def x := 0
```

What does this do?

1. Define `x` to be another name for the value `0`.

That's all!

What if we... don't?

Functional programming languages do not create places for you.

```
def x := 0
```

What does this do?

1. Define `x` to be another name for the value `0`.

That's all!

- `x` is not a place, and does not store a value.
- `x := 5` (i.e. `0 := 5`) does not make sense.

What if we... don't?

Functional programming languages do not create places for you.

```
def x := 0
```

What does this do?

1. Define `x` to be another name for the value `0`.

That's all!

- `x` is not a place, and does not store a value.
- `x := 5` (i.e. `0 := 5`) does not make sense.

Idea 1: Functional programming is programming that focuses on *values*.

But... places!

FAQ: I like mutable variables! Can I still follow this course?

But... places!

FAQ: I like mutable variables! Can I still follow this course?

Creating and modifying places is a kind of *effect*.

Other effects: reading input, throwing an exception, generating a random number, making a web request...

But... places!

FAQ: I like mutable variables! Can I still follow this course?

Creating and modifying places is a kind of *effect*.

Other effects: reading input, throwing an exception, generating a random number, making a web request...

We call code that has no effects *pure*.

Imperative code relies on place modification effects, so it is rarely pure.

Functional code is often pure, with effects added where needed.

But... places!

FAQ: I like mutable variables! Can I still follow this course?

Creating and modifying places is a kind of *effect*.

Other effects: reading input, throwing an exception, generating a random number, making a web request...

We call code that has no effects *pure*.

Imperative code relies on place modification effects, so it is rarely pure.

Functional code is often pure, with effects added where needed.

Idea 2: Functional programming permits, but controls, effects.

Working with Values

So what can we do with values?

1. Make expressions: `3 * 2 + 2`.

Working with Values

So what can we do with values?

1. Make expressions: `3 * 2 + 2`.
2. Give names: `let x := 2; 3 * x + x`.

Working with Values

So what can we do with values?

1. Make expressions: `3 * 2 + 2`.
2. Give names: `let x := 2; 3 * x + x`.
3. Abstract: `fun x => 3 * x + x`

Working with Values

So what can we do with values?

1. Make expressions: `3 * 2 + 2`.
2. Give names: `let x := 2; 3 * x + x`.
3. Abstract: `fun x => 3 * x + x`

When we abstract, we get a new type of value: a function.

- `3 * 2 + 2 : Nat`
- `fun x => 3 * x + x : Nat → Nat`

Working with Values

So what can we do with values?

1. Make expressions: `3 * 2 + 2`.
2. Give names: `let x := 2; 3 * x + x`.
3. Abstract: `fun x => 3 * x + x`

When we abstract, we get a new type of value: a function.

- `3 * 2 + 2 : Nat`
- `fun x => 3 * x + x : Nat → Nat`

Important distinction:

Value the result of a computation

Expression the program code describing a computation

Working with Functions

There are two important operations for functions:

Working with Functions

There are two important operations for functions:

- Abstraction: turn an expression into a function.
  - e.g. `fun x => 3 * x + x`

Working with Functions

There are two important operations for functions:

- Abstraction: turn an expression into a function.
  - e.g. `fun x => 3 * x + x`
- Application: apply a function to a value.
  - e.g. `(fun x => 3 * x + x) 2`

Working with Functions

There are two important operations for functions:

- Abstraction: turn an expression into a function.
  - e.g. `fun x => 3 * x + x`
- Application: apply a function to a value.
  - e.g. `(fun x => 3 * x + x) 2`

The first functional language was the λ-calculus, where these were the only two operations. The λ-calculus is Turing-complete.

Types

In Lean, every expression must have a type, which we denote `t : T`.

For example:

- `5 + 3 : Nat`
- `fun n => n + 3 : Nat → Nat`
- `(fun n => n + 3) 5 : Nat`

Often, the compiler can figure it out for you.

Types

In Lean, every expression must have a type, which we denote `t : T`.

For example:

- `5 + 3 : Nat`
- `fun n => n + 3 : Nat → Nat`
- `(fun n => n + 3) 5 : Nat`

Often, the compiler can figure it out for you.

Lean is special in that types are also values, and so also have a type.

For example:

- `Nat : Type`, `Nat → Nat : Type`
- `Type : Type 1`

Logic

In the first weeks of the course, we will focus on *propositions*: types whose values are proofs, written `A : Prop`.

Lean treats any two proofs of the same proposition as equal, so all that matters about a proposition is whether it has a value at all.

Logic

In the first weeks of the course, we will focus on *propositions*: types whose values are proofs, written `A : Prop`.

Lean treats any two proofs of the same proposition as equal, so all that matters about a proposition is whether it has a value at all.

There are two important propositions:

`True` the type with one value.

`False` the type with no values.

Logic

In the first weeks of the course, we will focus on *propositions*: types whose values are proofs, written `A : Prop`.

Lean treats any two proofs of the same proposition as equal, so all that matters about a proposition is whether it has a value at all.

There are two important propositions:

`True` the type with one value.

`False` the type with no values.

Note that when `A, B : Prop`, then also `A → B : Prop`.

Logic

In the first weeks of the course, we will focus on *propositions*: types whose values are proofs, written `A : Prop`.

Lean treats any two proofs of the same proposition as equal, so all that matters about a proposition is whether it has a value at all.

There are two important propositions:

`True` the type with one value.

`False` the type with no values.

Note that when `A, B : Prop`, then also `A → B : Prop`.

Idea 3: functions between propositions are implications.

Proofs

We can see the elements of a proposition `A : Prop` as its proofs.

- If there are no elements: the proposition is false.
- If there is an element: the proposition is true.

So functions of type `A → B` are proofs of `A → B`.

Proofs

We can see the elements of a proposition `A : Prop` as its proofs.

- If there are no elements: the proposition is false.
- If there is an element: the proposition is true.

So functions of type `A → B` are proofs of `A → B`.

Idea 4: Types can express propositions.

Idea 5: Terms can express proofs.

Lean 4 is a *functional programming language*
and *interactive theorem prover*.

Lean 4 is a *functional programming language*
and *interactive theorem prover*.

Functional programming language: you can define programs that operate on values.

Lean 4 is a *functional programming language*
and *interactive theorem prover*.

Functional programming language: you can define programs that operate on values.

Interactive theorem prover: you can reason about values and operations on those values.

Lean 4 is a *functional programming language*
and *interactive theorem prover*.

Functional programming language: you can define programs that operate on values.

Interactive theorem prover: you can reason about values and operations on those values.

Together: you can write a program and prove things about it.

Important note: we use Lean **4**.
Lean 3 is very different!

Who uses Lean?

Lean is primarily popular amongst mathematicians.

- Large amounts of mathematics are available in Mathlib.
- LLMs + Lean have gotten gold on the IMO.

Who uses Lean?

Lean is primarily popular amongst mathematicians.

- Large amounts of mathematics are available in Mathlib.
- LLMs + Lean have gotten gold on the IMO.

Lean is also good for cases when you need correctness.

- Amazon used Lean to develop Cedar.
- Microsoft used Lean to verify post-quantum cryptography in SymCrypt.

Logistics

Course plan

Proofs → Programs → Applications

You will learn:

Proofs How to express and prove things in Lean.

Programs How to express computations in Lean.

- And how to prove things about them.

Applications How to go from toy examples to an actual program.

Weekly rhythm

Each week:

- A lecture,
  - Online or offline, will be announced beforehand,
  - Mostly live-coding.

Weekly rhythm

Each week:

- A lecture,
  - Online or offline, will be announced beforehand,
  - Mostly live-coding.
- A seminar,
  - 10-minute quiz, then
  - More material in a more interactive setting.
- A homework assignment,
  - Organized via the LMS,
  - Submission via GitHub.

Rules and Expectations

Learning a language requires regular, focused work.

In particular:

- The lectures are us constructing things together.
  - Interrupt to ask questions if you have them.
  - Don't expect to be able to follow it while you're on your phone.
- The homeworks are there for you to practice writing it yourself.

Rules and Expectations

Learning a language requires regular, focused work.

In particular:

- The lectures are us constructing things together.
  - Interrupt to ask questions if you have them.
  - Don't expect to be able to follow it while you're on your phone.
- The homeworks are there for you to practice writing it yourself.

You may use as much AI as you want for the homework assignments. However, you are unlikely to develop the skills needed for the exam unless *you* write Lean code.

Assessment

Your grade has two components.

50% coursework Autograded weekly homework.

50% exam written exam, plus up to 10% bonus from quizzes.

You need at least 45% on **each** component to pass.

Homework

- Homework repos are distributed via the LMS.
- Check your own score locally: `./grade.sh`.
- Submit by pushing your repo.
- Your grade arrives after the deadline.

First two homeworks:

- HW00 is a trivial exercise to check your setup (due Friday).
- HW01 is the first real homework (due Tuesday evening).

Contact

Your options:

- Preferred: message `@kameliya_elektronika_bot` on Telegram.
  - I still see your message, but I get a TODO + FAQs get answered.
- Email: `kameliya.golova@jetbrains.com`
- University email: `kgolova@constructor.university`
  - I only check this occasionally, prefer the other email.

Contact

Your options:

- Preferred: message `@kameliya_elektronika_bot` on Telegram.
  - I still see your message, but I get a TODO + FAQs get answered.
- Email: `kameliya.golova@jetbrains.com`
- University email: `kgolova@constructor.university`
  - I only check this occasionally, prefer the other email.

Teaching assistants:

- Yuliya Karalenka
- Olesia Mutaeva
- Stanislau Palyn

Telegram groups

Announcements

Chat

Materials

If you want to know more about Lean, here are some good resources:

- lean-lang.org
- Theorem Proving in Lean 4
- Functional Programming in Lean
- Learn Lean 4